Graph

A few points:

So far, we have discussed linear data structures like list, stack, queue, circular list and doubly linked list.

We have also studied non linear structure tree and its variations.

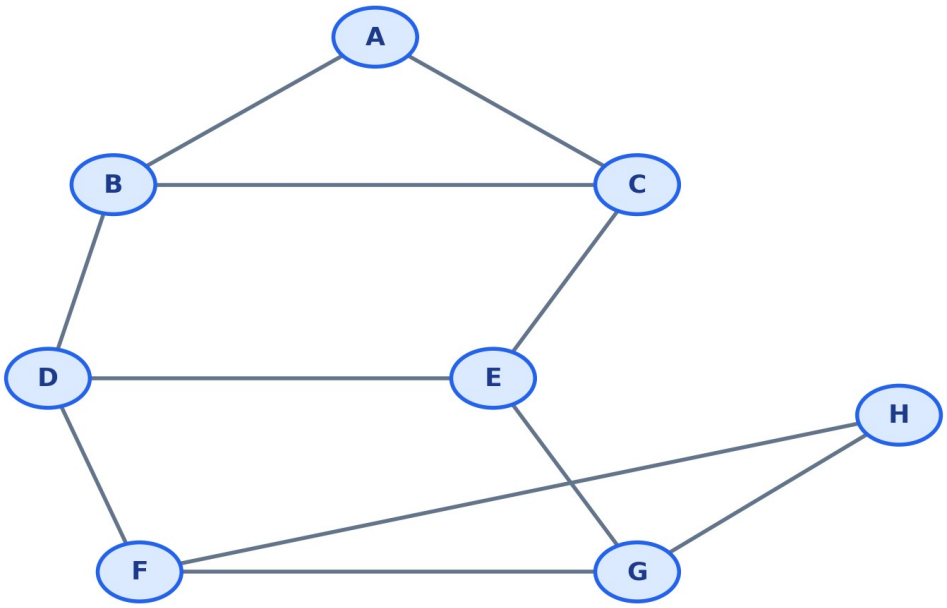
We shall discuss a new data structure called the graph

A **graph** is a **non-linear data structure** used to represent relationships between objects. It consists of vertices (nodes) and edges connecting pairs of vertices.

G = (V, E)

V is the set of vertices and E is the set of edges.

Let us take an example.



There are 8 vertices or nodes.

V = { A, B, C, D, E, F, G, H}

There are 11 edges.

AB, AC, BC, BD, CE, DE, DF, FG, EG, GH, FH

A graph may be directed or non-directed. The above graph is not directed.

Each edge may have some value associated with it – called weight. Such a graph is a weighted graph.

It could be the distance between a pair of geographical locations represented by the vertices, time or cost, …

Terminolgies :

- **Vertex (node):** An object represented in the graph, such as A.
- **Edge:** A connection between two vertices, such as AB.
- **Adjacent vertices:** Vertices connected directly by an edge; A and B are adjacent.
- **Incident edge:** An edge is incident on each endpoint; AB is incident on A and B.
- **Degree:** The number of edges incident on a vertex in an undirected graph.
- **In-degree:** The number of directed edges entering a vertex.
- **Out-degree:** The number of directed edges leaving a vertex.
- **Path:** A sequence of vertices joined by edges, such as A–B–D–F–H.
- **Path length:** The number of edges in a path; A–B–D–F–H has length 4.
- **Cycle:** A path returning to its starting vertex without repeating intermediate vertices, such as A–B–C–A.
- **Connected graph:** An undirected graph where every vertex is reachable from every other vertex.
- **Connected component:** A maximal connected part of a graph.
- **Simple graph:** A graph without self-loops or multiple edges between the same pair of vertices.
- **Self-loop:** An edge connecting a vertex to itself.
- **Complete graph:** A simple undirected graph where every pair of distinct vertices is connected.
- **Subgraph:** A graph formed from a subset of another graph's vertices and edges.
- **Cyclic / acyclic graph:** A graph that contains / does not contain a cycle.

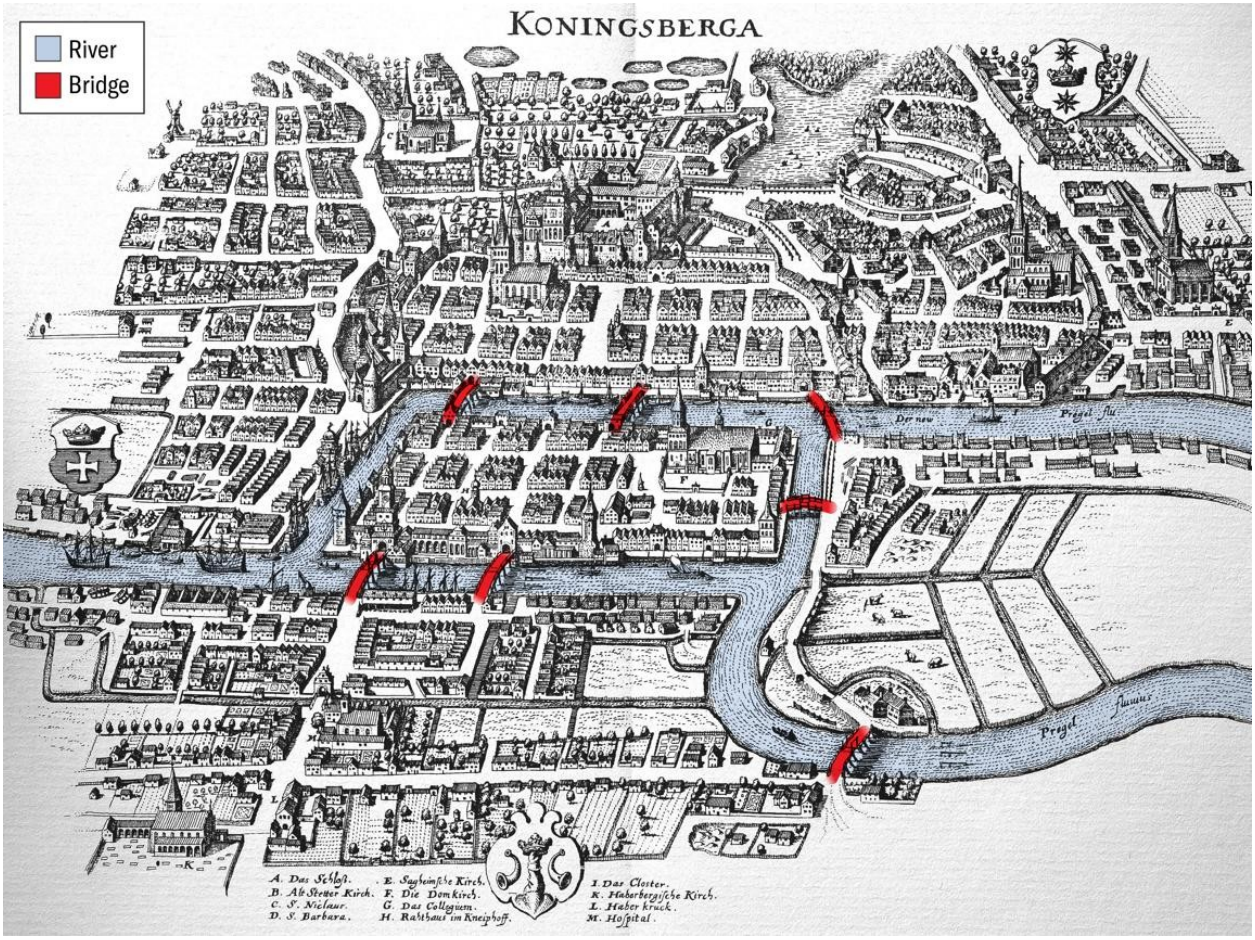
In this graph, we observe the following.

- Number of vertices: $|V| = 8$.
- Number of edges: $|E| = 11$.
- Neighbours of A: B and C.
- Degree of A: $\deg(A) = 2$.
- Degree of B: $\deg(B) = 3$ (edges AB, BC and BD).
- One path from A to H is A–B–D–F–H; its length is 4.
- A cycle is A–B–C–A.

- The graph is connected because every vertex can be reached from every other vertex.

For an undirected graph, $\Sigma \deg(v) = 2|E|$ because every edge contributes one degree to each endpoint. Here the degree sum is $22 = 2 \times 11$.

Königsberg Bridge Problem, Euler Graphs and Hamiltonian Graphs



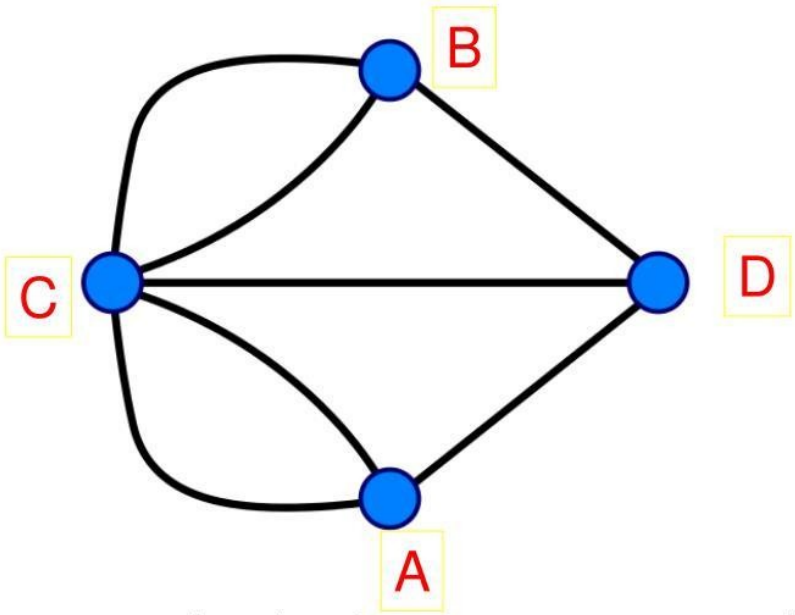
Königsberg Bridge Problem is a classical problem in graph theory.

There are 7 bridges across the rivers. The challenge is to start from any point, traverse the 7 bridges once and only once and return to the starting point. We cannot take any other means of travel.

This problem is supposed to have remained unsolved for a long time until Euler proved that it is not possible.

The simplified bridge map may be shown as in the next page. The requirement is draw the diagram without lifting the pen.

The seven bridges of Konigsberg =



Can you start and end at the same vertex, traversing every edge only once?

Euler's solution is simple. If we start from a vertex and end there, we require two edges. We enter vertex and leave it each time we require two edges. So, only if the degree of each vertex is even, it is possible to traverse a graph without repeating an edge.

A graph is said to be Eulerian if it supports the traversal of edges once and only once and come back to the starting point. The necessary and sufficient condition is that the degree of each node should be even.

What if want to visit each vertex only once and come back to the starting point? This is called Hamiltonian cycle. Even though this appears similar to Euler cycle, till today we do not have a simple solution to determine whether a Hamiltonian circuit exists for a given graph.

Applications of graphs:

- **Road and transport networks:** Vertices are cities or intersections; edges are roads. Weights can represent distance or travel time.
- **Social networks:** Vertices are people; edges are friendships, follows or interactions.
- **Computer networks:** Vertices are computers or routers; edges are communication links.
- **Course prerequisites:** A directed edge can mean one course must be completed before another.
- **Software dependencies:** Vertices are modules, files or tasks; edges represent dependencies.
- **Airline and logistics networks:** Vertices are airports or warehouses; edges represent routes.
- **Web pages:** Vertices are pages and directed edges represent hyperlinks.
- **Project scheduling:** Vertices represent tasks and edges represent precedence constraints

Abstract Data Type:

Operation	Purpose
create_graph()	Create an empty graph.
add_vertex(v)	Insert a vertex.
remove_vertex(v)	Remove a vertex and its incident edges.
add_edge(u, v)	Add an edge between u and v.
remove_edge(u, v)	Remove an edge.
has_vertex(v)	Check whether v exists.
has_edge(u, v)	Check whether an edge exists.
get_neighbors(v)	Return vertices adjacent to v.
degree(v)	Return the degree of v in an undirected graph.
display_graph()	Display vertices and edges.
destroy_graph()	Release resources used by the graph.

If the graph is directed, we should consider the direction. Adding an edge u to v will not be same as adding an edge v to u.

If the graph is weighted, we may want to set and/or get weights from the edges.

Graph representation

A. Adjacency matrix

An adjacency matrix A is a V × V array. For an unweighted graph, A[i][j] is 1 when an edge exists and 0 otherwise. For an undirected graph, the matrix is symmetric. The 8-vertex example requires an 8 × 8 matrix.

Space: $O(V^2)$. Checking whether an edge exists takes O(1) time.

The space required is proportional to the square of the number of vertices.

The time to check whether there is an edge between the vertices i and j requires constant time – does not depend on the number of vertices or number of edges.

B. Adjacency list

An adjacency list stores a list of neighbours for each vertex. For the example graph:

A → B, C
B → A, C, D
C → A, B, E
D → B, E, F
E → C, D, G
F → D, G, H
G → E, F, H
H → F, G
Space required will be a node for each vertex and another node for each edge.

There are other ways of representing the graph – we shall not discuss in this course.